Forge: A Modular Financial Reasoning Engine

Design, implementation, and independent accuracy validation across versions v0.1 and v0.2

Abhijeet Singh

Technical report generated 2026-10-13 from repository commit **67c09ff** · evaluator 1.0.3, oracle 1.1.0, rubric 1.1.0, dataset 1.2.0

Abstract

Forge is a modular financial reasoning engine: it ingests structured financial data (CSV and Excel), normalises it into a single domain model, analyses it through independent pure functions, and explains *why* an outcome occurred by citing the source data behind every claim. This report documents two released versions — **v0.1**, the command-line engine, and **v0.2**, a local web interface over the same pipeline — and the independent accuracy validation framework subsequently built to measure what the engine actually achieves rather than what it was assumed to achieve. The framework comprises an exact-rational reference implementation (an *oracle*) sharing no code with the engine, frozen development and held-out datasets under cryptographic hash, a written labelling rubric, and twelve separately-reported measures spanning numerical correctness, classification, insight precision and recall, evidence grounding, data-safety abstention, and structural invariants. Three measurement generations were conducted. The first established that the unmodified engine produced 93 critical failures on the held-out split and passed only 71.8% of cases on its primary numerical measure; 17 defects were identified and corrected. Two later generations resolved four outstanding behavioural decisions and added fixtures resembling real bank statements. At the current measurement the engine records zero critical failures, 147/148 = 99.3% on the primary numerical measure and 70/70 = 100.0% insight F_1. We argue that these figures do not support a blanket accuracy claim, identify the measures whose confidence bounds fall short of the project's 98% threshold, and state the limitations — single-labeller bias, synthetic data, and an unresolved merchant-vocabulary ceiling — that the benchmark cannot address.

1. Introduction

Tools that summarise personal or portfolio finances are common; tools that explain their own conclusions are rarer. Forge was built around a single constraint: **no insight without evidence**. Every figure the system reports must be traceable to the rows that produced it, and every qualitative claim must name the quantity that justifies it. This constraint shapes the architecture far more than any individual metric does.

The project has three design commitments that recur throughout this report. First, **analysis modules are pure functions** of the form `(model) => AnalysisResult`, performing no I/O and holding no state, which makes each one independently testable. Second, **interfaces render and never compute**: the command-line client and the browser client consume the same report object, so neither can introduce a figure the engine did not derive. Third, **the system refuses to guess**: an ambiguous input is surfaced as an assumption or rejected with a recorded reason, and a measure whose inputs are absent is reported as unavailable rather than as zero.

Section 2 states the scope boundary. Section 3 describes the architecture and the engine-interface contract. Sections 4 and 5 document what was built in v0.1 and v0.2 respectively. Section 6 sets out the validation methodology, Section 7 the measured results across three generations, Section 8 the threats to their validity, and Section 9 what is reproducible and how.

2. Scope and non-goals

The project maintains an explicit written boundary, and a change of scope is required to be recorded in the same change that implements it. In scope: ingestion of CSV and Excel exports; normalisation of heterogeneous column names, date orders, sign conventions and currencies into one model; seven analysis modules; an explainable reasoning layer; scenario simulation; and two local interfaces. Explicitly out of scope: any deployed or multi-user application, a public REST API, live bank connections, streaming ingestion, a persistent database, authentication, OCR of scanned statements, and machine-learning model training.

Two non-goals are worth stating as principles rather than exclusions. Forge does not give regulated financial advice; it assists understanding. And Forge does not replace a financial professional. These are not disclaimers bolted on at the end — they are why the reasoning layer is constrained to statements it can evidence.

3. Architecture and the engine-interface contract

The pipeline is a single directed path with no cycles and no shared mutable state:

```
ingest → detect shape → normalise → analyse → reason → (optionally) simulate → render
```

Ingestion reads CSV and `.xlsx` through a single dependency and detects whether a table holds transactions or portfolio holdings. **Normalisation** maps column aliases, infers date order, resolves sign conventions, reads currency from the amount cell where present, and records a per-row reason for every row it declines to accept. The result is a *financial model* that retains back-references to its source rows, which is what makes later traceability possible rather than aspirational.

Analysis comprises seven independent pure modules: cash flow, expenditure, trend, anomaly detection, asset allocation, portfolio concentration, and a composite financial-health assessment. Each returns metrics, findings and evidence. Currency integrity is enforced at the analysis boundary rather than during normalisation: normalisation reports what it saw, and analysis refuses a mixed-currency model outright, because no exchange-rate conversion is in scope.

Reasoning correlates analysis results into insights that each carry a rule identifier, a basis, a confidence, and tagged evidence items. **Scenario simulation** re-runs the pipeline under a hypothetical income change, a one-off expense or a recurring expense, and reports deltas — omitting any delta whose baseline is undefined, rather than computing it against an invented zero.

The boundary between engine and interface is a single documented object. Both interfaces consume it; the web client additionally passes through one pure presentation adapter that decides all formatting, tone and scaling server-side, so the browser renders finished strings. The practical consequence is that a presentation bug cannot become a numerical bug.

4. Version v0.1 — the engine and command-line interface

v0.1 delivered the complete pipeline behind a command-line interface with four subcommands (`analyze`, `inspect`, `scenario`, and later `serve`), multi-file ingestion, and machine-readable JSON output. The stack is TypeScript on Node with ECMAScript modules; the runtime dependencies are a CLI argument parser, a spreadsheet reader and a terminal colour library.

The architectural decisions taken during v0.1 and its subsequent hardening pass are recorded as a numbered decision register. Those most consequential for the results in Section 7 are summarised below.

ID	Decision	Consequence
D5	Analysis modules are pure functions	Each module is testable in isolation; the evaluation framework can call them directly.
D6	The model retains raw back-references	Evidence can cite a source row rather than restate a computed figure.
D7	No FX conversion; mixed currencies are refused	A mixed-currency dataset raises an error instead of silently summing incompatible amounts.
D8	Health score assesses only supported dimensions, and severe risks cap it	A critical weakness cannot hide behind strong dimensions. The cap threshold later proved to disagree with the code (see D21).
D11	Recurring commitments are excluded from anomaly scoring	A fixed cost such as rent cannot mask a genuine one-off. Initially matched identical amounts only (see D22).
D15	A typed error class for all user-facing failures	Both interfaces can render a clean message instead of a stack trace.

Table 1: Selected v0.1 design decisions and their downstream consequences.

5. Version v0.2 — a local web interface over the same engine

v0.2 added a browser interface, started from the command line, that uploads a file, renders the report across nine screens, and runs scenario simulations. The scope boundary was widened in the same change that implemented it, and deliberately narrowly: the server binds the loopback interface by default, has no accounts and no authentication, holds reports in a bounded in-memory cache for the lifetime of the process, stages uploads in a temporary directory and deletes them once the model is built, and adds **no new runtime dependency** — it is the standard library HTTP server plus static assets.

The governing decision was that the interface is a renderer and not a second engine. All presentation logic lives in one pure, unit-tested adapter; the browser computes nothing. This mattered immediately, because the visual design the interface was built from assumed data the engine does not produce. Rather than fabricate it, the interface shows what exists:

The design assumed	The engine provides	Resolution
Severities critical / high / medium / low	critical / warning / notice / info	Engine severities used; the design's colour slots reused.
An evidence table with a prior-period column	Evidence has no prior-period value	Column replaced. Inventing a prior value would be a fabricated figure.
Steps: Evidence / Cause / Impact	An explanation and named drivers	Relabelled Evidence / Drivers / Traceability; there is no separate impact model.
Hard-coded warnings and empty states	Skipped-row reasons, anomaly and trend evidence	Derived from the report, including an explicit “not assessed” screen.

Table 2: Where the visual design outran the engine. Each row is a case in which the interface was made less complete rather than more speculative.

A missing measure — no health score, no holdings, no anomalies — is stated as unavailable together with the engine's own reason, never as a zero. This is the same principle that later produced defect AV-17, where scenario deltas had been computed against a fabricated baseline of zero.

6. Methodology: independent accuracy validation

The engine's accuracy had been assumed rather than measured. The validation framework was therefore built to answer a narrower and more answerable question: on a frozen set of cases whose correct answers were established independently of the engine, how often is the engine right, and where it is wrong, how wrong?

6.1 The oracle

A reference implementation computes every expected value in exact rational arithmetic using arbitrary-precision integers, with inputs given as exact decimal strings. It imports nothing from the engine's source — a test enforces this — so agreement between the two is genuine corroboration rather than a tautology. Its limit is equally important: the oracle is an independent *implementation*, but its *specification* (formulas, constants, weights) was transcribed from the engine's source into a written algorithm reference by the same author. It can therefore detect a deviation from the stated design, but not a mistaken design.

This limitation has a useful consequence. Because the oracle follows the written specification, a decision to change an algorithm must be written into the specification first and transcribed second; when new engine code disagrees with the oracle, that is the instrument working rather than failing.

6.2 Splits, freezing and leakage control

Cases are divided into a **development** split, which may be inspected and debugged against, and a **held-out** split used for measurement only. Both are frozen JSON files whose SHA-256 digests are recorded in a manifest; the evaluator refuses to run if a digest differs, and a unit test re-checks the manifest. The held-out split shares no case identifier, transaction set or classification string with development.

The order of work was: write the oracle and the development set and debug on development only; write the held-out set before any held-out result existed; freeze both digests; measure the baseline; fix the engine; measure again. No engine change was motivated by a held-out failure. Where a held-out baseline failure revealed a bug in the *instrument*, the instrument was corrected, the change disclosed, and the baseline re-measured.

A digest proves the data has not silently changed. It does not prove the data resembles real users, and a held-out split reduces benchmark overfitting without eliminating it. Section 8 returns to this.

6.3 Measures

Scores are never blended into a single number. Each measure has its own denominator and its own meaning, and each is reported with a Wilson score interval (bootstrap for F_1). Critical failures — corrupted totals, silently accepted garbage, fabricated values, mixed currencies, non-finite output — are counted separately so that a high average cannot conceal one.

ID	Measure	What it counts	Kind
A1	Numerical correctness — per comparison	Fraction of oracle-eligible metric comparisons within the predefined tolerance.	mathematical
A2	Numerical correctness — per case	Fraction of cases in which EVERY numeric and availability expectation is met.	mathematical
A3	Numerical correctness — per case, unambiguous cases only	As A2, excluding cases that rest on a documented interpretation where another is valid.	mathematical
A4	Cell parsing — correct or safely rejected	Fraction of raw amount/date cells read correctly or rejected with a recorded reason.	human-labelled
A5	Categorisation (keyword heuristic)	Fraction of descriptions mapped to the human-labelled category.	human-labelled
A6	Asset-class mapping (keyword heuristic)	Fraction of asset-class labels mapped to the human-labelled class.	human-labelled
B1	Insight correctness — micro F1	F1 over every graded finding-type label (rules, anomalies, trend rises, cash-flow and spending findings).	human-labelled
B2	Insight labels — accuracy	(TP+TN) / graded labels. Reported with a Wilson interval because the F1 bootstrap interval collapses to a point when every label is correct.	human-labelled
C1	Evidence grounding — observed facts	Fraction of insight evidence items that exist in the report AND whose entity, period and value match the source data, plus numbers quoted in insight text.	mathematical
C2	Evidence grounding — attribution claims	Fraction of heuristic attributions (driver, one-off, reserves) that the source data supports.	rule-specific

ID	Measure	What it counts	Kind
D1	Data safety and abstention	Fraction of safety expectations met (errors, warnings, skipped rows with reasons, refusal to guess, unavailable measures).	mathematical
D2	Invariants	Fraction of invariant checks holding across every successfully analysed case.	mathematical

Table 3: The twelve reported measures, with definitions read directly from the evaluator's own output. A2 is the primary numerical score: a case counts only if *every* numeric and must-be-unavailable expectation in it is met. A3 is A2 restricted to cases with a single defensible reading.

6.4 Tolerances

Currency comparisons use a tolerance four orders of magnitude below one cent, so any wrong cent fails; a test asserts that a one-cent error on a billion-unit total still fails. Ratios use a relative tolerance of 1e-10. Counts and the 0–100 health score are compared exactly. Numbers appearing in prose are compared at half a unit of their printed precision.

7. Results

Three generations of measurement were conducted. They are reported separately and never combined: between generations the rubric, dataset, evaluator or oracle version changed, so the comparison tool refuses to table them together. Every change between generations made the benchmark harder or its denominators larger; none relaxed a tolerance or an expected value. The accuracy threshold the project reports against is 98.0% — a threshold, never a result.

Gen.	Dataset	What it measured	Held-out cases	Baseline commit	Final commit
I	1.0.0	The AV-01..AV-17 defect audit	153	60c1308	60c1308
II	1.1.0	Decisions D21-D23	156	cc9d6a9	ae34cce
III	1.2.0	Realistic statement shapes; decision D24	159	49c5dc9	f437a4c

Table 4: The three measurement generations. Case counts are read from the frozen datasets. Generation I shows the same commit twice because it predates the commit discipline described in Section 9: both of its reports were taken from an uncommitted working tree and record the repository's initial commit. Its baseline was later reproduced byte-for-byte from a dedicated framework commit; Generations II and III were measured at distinct commits by construction.

7.1 Generation I — dataset 1.0.0: The AV-01..AV-17 defect audit

ID	Measure	n	Baseline	Final	95% CI (final)	Lower bd. ≥ target
A1	Numeric — per comparison	5605	99.6%	**99.9%**	[99.8%, 100.0%]	yes
A2	Numeric — per case	142	71.8%	**97.9%**	[94.0%, 99.3%]	no
A3	Numeric — per case, unambiguous cases only	135	73.3%	**100.0%**	[97.2%, 100.0%]	no
A4	Cell parsing — correct or safely rejected	34	55.9%	**100.0%**	[89.8%, 100.0%]	no
A5	Categorisation (keyword heuristic)	24	87.5%	**87.5%**	[69.0%, 95.7%]	no
A6	Asset-class mapping (keyword heuristic)	19	68.4%	**84.2%**	[62.4%, 94.5%]	no
B1	Insight correctness — micro F1	61	90.9%	**100.0%**	[100.0%, 100.0%]	no
B2	Insight labels — accuracy	61	93.4%	**100.0%**	[94.1%, 100.0%]	no
C1	Evidence grounding — observed facts	1294	99.4%	**99.7%**	[99.2%, 99.9%]	yes
C2	Evidence grounding — attribution claims	40	100.0%	**100.0%**	[91.2%, 100.0%]	no
D1	Data safety and abstention	449	96.9%	**100.0%**	[99.2%, 100.0%]	yes
D2	Invariants	2238	100.0%	**100.0%**	[99.8%, 100.0%]	yes

Table 5: Held-out results, dataset 1.0.0. Critical failures: baseline 93, final 0. Total failing checks across both splits: baseline 300, final 25.

7.2 Generation II — dataset 1.1.0: Decisions D21-D23

ID	Measure	n	Baseline	Final	95% CI (final)	Lower bd. ≥ target
A1	Numeric — per comparison	5692	99.8%	**100.0%**	[99.9%, 100.0%]	yes
A2	Numeric — per case	145	91.0%	**100.0%**	[97.4%, 100.0%]	no
A3	Numeric — per case, unambiguous cases only	139	90.6%	**100.0%**	[97.3%, 100.0%]	no

ID	Measure	n	Baseline	Final	95% CI (final)	Lower bd. ≥ target
A4	Cell parsing — correct or safely rejected	34	100.0%	**100.0%**	[89.8%, 100.0%]	no
A5	Categorisation (keyword heuristic)	24	87.5%	**100.0%**	[86.2%, 100.0%]	no
A6	Asset-class mapping (keyword heuristic)	19	84.2%	**100.0%**	[83.2%, 100.0%]	no
B1	Insight correctness — micro F1	70	91.7%	**100.0%**	[100.0%, 100.0%]	no
B2	Insight labels — accuracy	70	94.3%	**100.0%**	[94.8%, 100.0%]	no
C1	Evidence grounding — observed facts	1306	99.7%	**100.0%**	[99.7%, 100.0%]	yes
C2	Evidence grounding — attribution claims	41	100.0%	**100.0%**	[91.4%, 100.0%]	no
D1	Data safety and abstention	458	100.0%	**100.0%**	[99.2%, 100.0%]	yes
D2	Invariants	2274	100.0%	**100.0%**	[99.8%, 100.0%]	yes

Table 6: Held-out results, dataset 1.1.0. Critical failures: baseline 0, final 0. Total failing checks across both splits: baseline 51, final 1.

7.3 Generation III — dataset 1.2.0: Realistic statement shapes; decision D24

ID	Measure	n	Baseline	Final	95% CI (final)	Lower bd. ≥ target
A1	Numeric — per comparison	5777	99.8%	**99.9%**	[99.7%, 99.9%]	yes
A2	Numeric — per case	148	99.3%	**99.3%**	[96.3%, 99.9%]	no
A3	Numeric — per case, unambiguous cases only	140	99.3%	**99.3%**	[96.1%, 99.9%]	no
A4	Cell parsing — correct or safely rejected	34	100.0%	**100.0%**	[89.8%, 100.0%]	no
A5	Categorisation (keyword heuristic)	24	100.0%	**100.0%**	[86.2%, 100.0%]	no
A6	Asset-class mapping (keyword heuristic)	19	100.0%	**100.0%**	[83.2%, 100.0%]	no
B1	Insight correctness — micro F1	70	100.0%	**100.0%**	[100.0%, 100.0%]	no
B2	Insight labels — accuracy	70	100.0%	**100.0%**	[94.8%, 100.0%]	no
C1	Evidence grounding — observed facts	1318	100.0%	**100.0%**	[99.7%, 100.0%]	yes
C2	Evidence grounding — attribution claims	44	100.0%	**100.0%**	[92.0%, 100.0%]	no
D1	Data safety and abstention	467	100.0%	**100.0%**	[99.2%, 100.0%]	yes
D2	Invariants	2310	100.0%	**100.0%**	[99.8%, 100.0%]	yes

Table 7: Held-out results, dataset 1.2.0. Critical failures: baseline 0, final 0. Total failing checks across both splits: baseline 25, final 21.

7.4 Interpretation

The first generation is the one that justified the exercise. Measured against the unmodified engine, the held-out split produced **93 critical failures** and a primary numerical score of 71.8%; cell parsing was correct or safely rejected only 55.9% of the time. Seventeen distinct defects were identified, each given a regression test verified to fail on the original code. They included silently wrong number parsing (a value of 1e3 read as 13), acceptance of impossible calendar dates, a degenerate anomaly statistic that reported a value five per cent above its cluster as “337,250,000× typical”, text encoded as UTF-8 but decoded as Latin-1, four reasoning rules that over-claimed, and scenario deltas computed against a fabricated baseline of zero.

The second generation resolved four behavioural questions that the first had deliberately left open rather than settling unilaterally, because each was a product decision and not a bug: whether the health-score cap fires at a bare majority of loss-making months or at a stricter threshold (the documentation and the code disagreed); whether ordinary large-category spending such as a monthly grocery bill should be reported as an anomaly; how to represent a savings rate when income is zero; and how to improve two keyword heuristics without tuning them against the benchmark.

The third generation added fixtures resembling real bank exports — abbreviated and inconsistent merchant strings with no category column, two accounts with own-account transfers, statements beginning and ending mid-month, instalment plans and partial refunds. The transfer and partial-month shapes passed. The merchant strings did not, and this is the most informative single result in the report: with no category column the engine recovers five of six categories, because a small set of brand names and abbreviations are absent from its keyword vocabulary. It *declines* rather than guessing wrong, which is the intended behaviour, but a user of a real statement would see a large uncategorised bucket.

That addition moved the primary numerical score from 100.0% on dataset 1.1.0 to 99.3% on dataset 1.2.0. **The fall is the finding, not a regression:** the benchmark became harder while the engine improved. A measure that only ever rises is measuring its own fixtures.

7.5 On the 98% threshold

The project treats 98.0% as a threshold to compare against, and permits the claim only for a named measure on a named dataset when both the point estimate and the lower confidence bound support it. At the current measurement, 4 of 12 held-out measures have a lower bound at or above the threshold: A1, C1, D1, D2. Each of those pools many correlated checks per case, so their intervals assume an independence they do not have and are optimistic. The per-case measures that the methodology directs the reader to prefer — A2 and A3 — have point estimates of 99.3% and 99.3% but intervals of [96.3%, 99.9%] and [96.1%, 99.9%]. **No blanket claim of 98.0% accuracy is supportable from this evidence**, and several measures with a point estimate of 100% have samples too small for their lower bound to reach it.

8. Threats to validity

The following are limitations of the evidence, not caveats about the engine. They are stated because a benchmark result is only as useful as the reader's ability to discount it correctly.

Synthetic data. Every case is synthetic by design, since the project forbids private data in fixtures. Dataset 1.2.0 narrowed the gap by representing merchant noise, multiple accounts, partial months, instalments and refunds, and that addition immediately found a limitation three earlier generations had missed. What remains absent: real-world text encodings, multi-currency households, institutions nobody on the project has seen, and the volume and irregularity of several years of one person's data. Six hand-written cases resembling real exports are not a sample of real exports.

Single labeller. All labels, the rubric and the oracle's specification come from one author who had read the engine's source. There was no second, blind reviewer, so label uncertainty is unquantified. The most recent cases are weaker still on this axis: they were authored after the fix they measure had been designed, by whoever designed it. A blind labelling pack and a reconciliation tool reporting agreement and Cohen's κ now exist, but no independent review has been conducted. This is the framework's largest remaining weakness.

Correlated checks. Four measures pool many checks per case. Their confidence intervals assume independence and are therefore optimistic. The per-case measures are the honest summary.

Small labelled samples. The labelled measures rest on a few dozen items each, spread across ten finding types, so per-type precision and recall rest on a handful of observations.

Weak checks exist and are reported as such. At least one grounding check passes whenever the rule's own guard holds, and many invariants hold by construction. They are reported separately so they cannot dominate a headline.

Accumulating benchmark pressure. Four open items have now been closed by changing the engine. Each change was specified before it was implemented and measured against an oracle that follows the specification, and none was motivated by a held-out failure — but a fixture set a system has been repeatedly fixed against is a weaker instrument each time. New held-out material, not more development cases, is what restores it.

Usefulness is not measured. The framework asks whether an insight is correct, supported and not over-claimed. It does not ask whether a person would act on it.

8.1 Standing limitations in the engine

Two behavioural limitations are deliberately kept visible rather than closed, and a consumer of the engine's output needs both.

Zero-income savings rate. A savings rate is undefined when income is zero. The engine emits the metric as zero accompanied by an informational finding stating that the zero means “not measurable”, because removing the key from the published contract would be a breaking change. Any consumer reading that field must therefore check for the accompanying finding before treating a zero as a measurement. The evaluator enforces the present behaviour — a bare zero with no explanation is a failure — so it cannot silently regress into a fabricated figure. Whether to withhold the key instead remains an open contract decision.

Cross-file duplicate transactions are not detected. Merging several files is deterministic but performs no deduplication, so a transaction appearing in two overlapping exports is counted twice, and every figure derived from it — inflow, outflow, net, category totals, anomaly baselines, the health score — is wrong by that amount. The limitation is *declared unsupported* within the benchmark and reported rather than scored, so no measured rate in Section 7 should be read as evidence about it. Users must merge non-overlapping periods or deduplicate before ingestion.

A third, newer limitation sits alongside them: the merchant-vocabulary ceiling described in Section 7. It is recorded with an explicit instruction not to close it by adding brand keywords, which would be vocabulary padding against cases already inspected and specific to one country's retailers. If merchant coverage matters, it needs a maintained list treated as data, or an explicit user-facing step, measured on fresh held-out cases.

9. Reproducibility

A measurement that cannot be re-derived is a recollection. Each baseline in this report was therefore taken at a commit that carries the framework, the frozen datasets and the specification but **no engine change**; the fixes follow in the next commit. Every report records the commit it was measured at and whether the working tree was clean, and the first generation's baseline was verified byte-for-byte against a fresh run from its commit — identical across every rate, denominator, failure record and the integrity block.

Artefact	Commit	Clean tree	Measured
Gen. I baseline (dataset 1.0.0)	60c1308	yes	2026-10-03
Gen. I final (dataset 1.0.0)	60c1308	no	2026-10-03
Gen. II baseline (dataset 1.1.0)	cc9d6a9	yes	2026-10-12
Gen. II final (dataset 1.1.0)	ae34cce	yes	2026-10-12
Gen. III baseline (dataset 1.2.0)	49c5dc9	yes	2026-10-12
Gen. III final (dataset 1.2.0)	f437a4c	yes	2026-10-12

Table 8: Provenance of every measurement in this report. The clean-tree column for Generation I should be read with suspicion rather than trust: the flag is produced by a probe that returns *false* when it fails, so a run performed outside the repository is indistinguishable from a clean one. That is precisely why the Generation I baseline was re-derived from a dedicated commit and compared byte-for-byte. One Generation II run was taken from a modified tree and was re-measured from a clean checkout, giving identical figures. A cross-check tool now asserts the property automatically, together with agreement on versions, dataset digests and every denominator across all four places a measurement is published.

The full set is reproduced with:

```
npm install
npm test # unit, integration, regression and property tests
npm run typecheck # engine and evaluation framework
npm run validate:accuracy -- --strict
npm run validate:accuracy:verify
python docs/paper/build_report.py # regenerates this document
```

The repository has 12 commits of history at **67c09ff**. At that commit the unit, integration, regression and property tests all pass, the type checker covers both the engine and the evaluation framework without error, the production build is clean, the accuracy run exits zero under `--strict`, and the artefact cross-check passes. The measurements reported here were produced on Node v22.22.1 (win32).

10. Conclusions and further work

Forge demonstrates that an explainability constraint taken seriously at the architectural level — pure analysis functions, a model that retains its source rows, interfaces that render rather than compute, and a refusal to report a figure the data does not support — is compatible with a system that also measures well on a benchmark it did not design around. The more transferable result is methodological. Building an independent oracle, freezing held-out data under hash, and ordering commits so that each baseline is reproducible turned a set of assumptions into a set of measurements, and the first measurement found seventeen defects in code that had passed its own test suite.

Three findings are worth carrying forward. First, a benchmark whose numbers only improve is measuring its own fixtures; the most valuable change made to it caused a headline score to fall. Second, instrument provenance deserves the same scepticism as results — a flag recording whether the source tree was clean turned out to fail open, so its reassuring value

meant nothing until a cross-check was written. Third, the limits that matter most are the ones a benchmark cannot see: single-labeller bias and synthetic data bound every figure in Section 7, and no amount of additional cases written by the same author will relax that bound.

Further work, in order of value: an independent labelling review, for which the apparatus now exists and only a second person is missing; a decision on the zero-income savings-rate contract; a maintained merchant vocabulary treated as data if real-statement categorisation matters; and cross-file duplicate detection, should it move into scope.

Appendix A. Decision register

Decisions are numbered and recorded at the time they are taken, together with the alternative that was rejected. The register below is abridged; identifiers are stable and refer to the project's decision document.

ID	Decision
D1–D4	TypeScript on Node; command-line interface first; ECMAScript modules; a single spreadsheet dependency.
D5–D6	Analysis modules are pure functions; the unified model retains raw back-references for traceability.
D7	Currency integrity: no exchange-rate conversion, and a mixed-currency model is refused at the analysis boundary.
D8	Health scoring assesses only dimensions whose inputs exist, with severe risks capping the headline score.
D9–D11	Evidence traceability; cash-flow semantics; an anomaly baseline that excludes recurring commitments.
D12–D15	Date-order inference; per-row skip reasons; provenance recording; a typed user-facing error class.
D16	The web interface is a renderer, not a second engine: all presentation logic is one pure, tested adapter.
D17–D20	An independent benchmark; strict parsing with abstention; unavailable is not zero in scenarios; reasoning rules must not over-claim.
D21	The health-score cap fires at a strict majority of loss-making months, resolving a documentation-versus-code conflict in favour of the documented rule.
D22	Regular spend in a recurring category is not an anomaly. Candidacy is narrowed; the baseline distribution is deliberately left pooled, because narrowing it would empty the baseline on ordinary ledgers.
D23	The categoriser matches on stated rules — token boundaries, longest match wins, and direction awareness — rather than on map order.
D24	A plural in a description matches a singular keyword, correcting a regression introduced by D23's token-boundary rule.

Appendix B. Defects found by the first measurement

Each defect below was found by the benchmark against the unmodified engine, and each has a regression test verified to fail on the original code. The list is the substance of the claim that the exercise was worth conducting.

ID	Observed behaviour before correction
AV-01	Number parsing returned silently wrong values: 1e3 became 13, a European-grouped 1.234,56 became 1.23456, a Unicode minus sign was read as positive.
AV-02	Impossible calendar dates were accepted (30 February, 31 April), and arbitrary text became a date through permissive parsing.
AV-03	A zero median absolute deviation was replaced by a near-zero constant, reporting a value five per cent above its cluster as 337,250,000 times typical.
AV-04	A reported multiple was the modified z-score rather than a multiple of the median.
AV-05	Zero income produced a silent savings rate of zero per cent.
AV-06	Currency symbols within amounts were ignored, so dollar and euro rows were summed as one currency.
AV-07	A row carrying both a debit and a credit silently dropped the debit.
AV-08	A holding whose value disagreed with quantity times price was accepted without comment.
AV-09	Fund and ETF keywords shadowed more specific asset classes, classifying a gold ETF as equity.
AV-10	UTF-8 input was decoded as Latin-1, corrupting currency symbols, merchant names and categories.
AV-11	A newly appearing spending category was reported as having risen one hundred per cent.
AV-12	An insight citing the largest one-off outflow in fact cited the largest outflow overall, often recurring rent.
AV-13	A portfolio-concentration insight named a cash position as the concentration driver.
AV-14	A healthy-position insight fired on one or two good months.
AV-15	A cash-flow-decline insight attributed the decline to rising spending when income had fallen and spending was flat.
AV-16	Scenario simulation accepted an income change below minus one hundred per cent, and non-finite inputs.
AV-17	Scenario deltas were computed against a fabricated baseline of zero, so runway appeared to improve after adding an expense.

All measured figures in this report were read programmatically from the committed measurement artefacts under evaluation/results/ at build time. No percentage was transcribed by hand.